



JS / JavaScript

[Ok ceci est un cours long ... Si vous connaissez déjà JavaScript, vous pouvez le lire rapidement et passer aux exercices et pensez à utiliser TypeScript.

Sinon, lisez jusqu'à (et y compris) la page 7 pour faire les questions 1-5 de Activity 3. Puis revenez pour lire la suite avant de passer aux questions suivantes de Activity 3.]

L'HTML ne dispose d'aucune intelligence: il n'est pas possible d'effectuer des calculs, il ne peut pas vérifier si quelqu'un a correctement rempli un formulaire, et il est incapable de prendre des décisions en fonction de la manière dont interagit le visiteur avec la page Web. De manière simple, nous pouvons dire que l'HTML permet de lire du texte, regarder des images et cliquer sur des liens pour se rendre sur d'autres pages Web avec plus de textes et d'images. Afin d'ajouter de l'intelligence aux pages Web de manière à ce qu'elles puissent répondre aux visiteurs, nous avons besoin de **JavaScript (JS)**.

Le JavaScript est un langage côté client, ce qui signifie qu'il est exécuté par le navigateur à l'inverse de langages comme le PHP qui sont généralement exécutés par le serveur. L'intérêt d'un langage exécuté côté serveur est de pouvoir accéder à des bases de données, envoyer des mails, etc. Le problème d'un langage exécuté côté serveur est que le navigateur doit envoyer une requête au serveur pour que ce dernier la prenne en compte. Donc un langage (comme JavaScript) exécuté côté client peut réagir immédiatement et modifier ce que l'utilisateur voit dans son navigateur Web, et cela sans avoir à forcément contacter le serveur ou charger une nouvelle page. C'est, par exemple, le cas de Google Maps, où quand une carte est chargée, il est possible de zoomer ou de se déplacer dessus, etc. et cela sans avoir besoin de recharger une page, le code JavaScript se charge d'interroger en arrière plan le serveur si nécessaire.

Aujourd'hui, le JavaScript est en plein essor car les moteurs d'exécution des navigateurs ont gagné en vitesse et également depuis l'avènement de l'HTML 5 pour lequel certaines fonctionnalités, comme le canvas, ne sont accessibles que par l'intermédiaire du JavaScript. N'importe quelle plate-forme (micro-ordinateur, smartphone, etc.) dispose d'un navigateur qui sera capable d'exécuter du JavaScript.

Remarques : En fait, le JavaScript peut également être utilisé du côté serveur. Par exemple, la plateforme Node.js, utilisée pour développer des applications Web, s'exécute du côté serveur (pour le plupart) et est programmée en JavaScript. Cela reste, cependant un cas particulier. Dernière chose, le JavaScript n'a aucun rapport, en dehors du nom, avec le langage Java (plus près du C).

Ajouter de JS

Une page peut déjà être interactive car elle peut être scrollable, les formulaires HTML (quoi? oui plus tard) peuvent être modifié/remplis et le CSS peut varier (par exemple avec le sélecteur `:hover` quand la souris rentre/sort par dessus un élément). Toutefois ces interactions ne permettent pas de faire des applications aussi interactives que des application dites natives (mac/linux/windows).

JavaScript va permettre de s'abonner à presque tous les évènements possibles engendrés par l'utilisateur (souris, clavier, accéléromètres) et d'y réagir. Dans cette partie nous allons surtout voir les entrées (évènements) car Javascript ne peut pas accéder au

(10 pages de contenu,

disque dur du client et/ou une base de donnée et comment modifier la page web (et donc le rendu) et dans la partie AJAX comment accéder au réseau.

Pour illustrer cette partie nous n'utiliserons que 2 sorties JavaScript :

- `console.log("Hello")` qui affiche du texte dans la console (en espérant que vous savez ouvrir et regarder la console de votre navigateur web) ;
- `alert("world")` qui ouvre une fenêtre modale, affiche le message et bloque l'exécution Javascript tant que l'utilisateur n'a pas cliqué "OK" ;
- `prompt("tapez")` qui ouvre une fenêtre modale pour l'utilisateur de taper.

Pour inclure du script javascript dans une page web 2 possibilités s'offrent à nous :

- inclure le tag `<script> console.log("hello"); </script>` n'importe où
- inclure le tag `<script src="main.js"> </script>` C'est mieux de privilégier cette méthode, c.à.d. le fichier externe car le navigateur le met (souvent) en cache et ne le recharge pas systématiquement. On peut ajouter le tag `<script>` n'importe où ...

Il est courant de mettre le script just avant le `</body>` pour être sûr que tous nos éléments de la page sont créés (ou entre `<head>...</head>` en haut de la page web si le script n'a pas besoin des valeurs des éléments de la page). Si vous voulez exécuter le script à la fin mais vous aimez mettre tous vos scripts / librairies tôt (vraiment question de style) vous pouvez ajouter le terme `defer` après le nom du script, Le script sera exécuté quand la page sera complètement affichée par le navigateur (en fait quand elle aura été complètement parsée par le navigateur). `<script src="script.js" defer></script>`

Un exemple de JavaScript

Voici notre première exemple en JavaScript, nous allons ajouter un script qui ouvre un dialogue pour demander un chiffre, fait un loop et affiche le résultat dans la page web.

D'abord la page web en html (à noter le id ajouté au paragraph `<p>` que nous allons modifier, et l'appelle au script `class-loop.js`).

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Je vais compter </title>
  </head>
  <body>
    <h1>Je compte jusqu'au </h1>
    <p id="counter"> Du texte à remplacer ...</p>

    <script src="class-loop.js"></script>

  </body>
</html>
```

Et notre script `class-loop.js`

```
let text = prompt('Tapez un numero pour compter : '); /* entrée utilisateur */
let c = parseInt(text,10); /* transformer text en integer */
let i = 0;

if( ! Number.isNaN(c) ) { /* verifier que c'est effectivement un integer */
```

```

    while (i < c) {
        console.log(i);
        ++i;
    }
} else {
    c = text + " n'est pas un numero! ";
}

document.getElementById('counter').innerHTML = c; /* changer l'html ! */

```

On a ici :

- des commentaires entre `/* */`
- le `prompt("question")` qui ouvre une fenêtre modale pour taper ;
- le mot clé `let` pour déclarer des variables (mais aiii, la variable `c` est d'abord un numero, puis du texte ...)
- la méthode `parseInt(text)` pour convertir texte à un numero ;
- une structures conditionnelle (`if`) pour faire un test, et des boucles (`while`) ;
- la méthode `Number.isNaN(c)` qui teste si `c` est un numero;
- `console.log("Hello")` qui affiche du texte dans la console ;
- et la méthode `document.getElementById('counter').innerHTML` pour accéder aux éléments de notre page html (et les changer).

Alors JavaScript c'est un langage de programmation !

Débogueur

Un script javascript (ici `class-loop.js`) ne PEUT PAS inclure un autre script javascript (enfin pas facilement et sans tricher) et donc les pages web pourront commencer par une grande ribambelle de scripts et de librairies à inclure (dans le bon ordre SVP !). Exemple :

```

<script src="libraries/p5.min.js"></script>
<script src="libraries/rhill-voronoi-core.min.js"></script>
<script src="libraries/simplex-noise.js"></script>
<script src="libraries/jquery-3.1.1.js"></script>

```

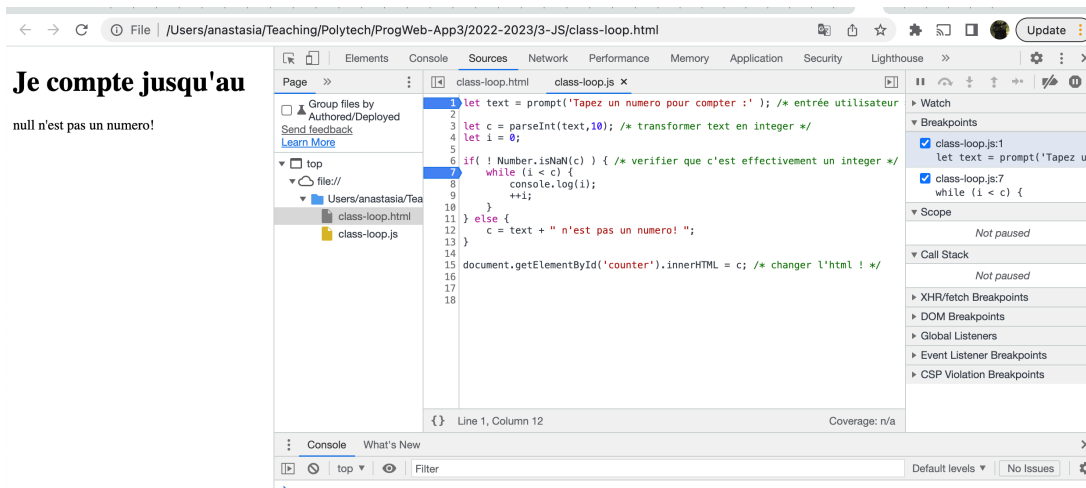
Bon le prochain concept à voir n'est pas évident MAIS important : JavaScript n'est pas multi-threadé mais le navigateur oui. Ca veut dire que dans l'exemple ci dessus le navigateur peut mettre plusieurs ressources à charger en même temps (des images, des scripts à télécharger) mais quand du code à l'intérieure d'une balise s'exécute AUCUN autre code ne s'exécute en même temps. Si un script a fini de se télécharger ou si un évènement arrive (ie click souris) il est mis en attente dans une liste de code à exécuter plus tard.

Corollaire: si vous faites une boucle infinie en JavaScript vous risquez freezer la page. Qu'est ce qu'on peut faire alors : (i) fermer l'onglet gelé; ou (ii) si la page n'est pas feezée ouvrir le gestionnaire de tache du navigateur (si si ça existe et ce n'est pas le gestionnaire de tache de l'OS) et tuez le script fautif. Et (iii) quand vous déboguez vous pouvez utiliser les outils de développement pour arrêter le script.

Quand vous inspectez la page, si les outils de développement sont ouverts dans l'onglet "Source" vous avez des boutons pour arrêter/repandre l'exécution et qui affiche en plus la stack qui permet de voir à quel endroit du code l'exécution est bloquée, de mettre de breakpoints, etc.

Bon voilà, il vous faudra manipuler un peu ce puissant outil de débogage qui vient avec votre navigateur Web mais ça sera essentiel pour progresser dans cette matière.

Voici une image des outils de développement :



Il nous reste 3 choses importantes à voir : le DOM, les événements auxquels peut réagir JavaScript et le langage lui-même.

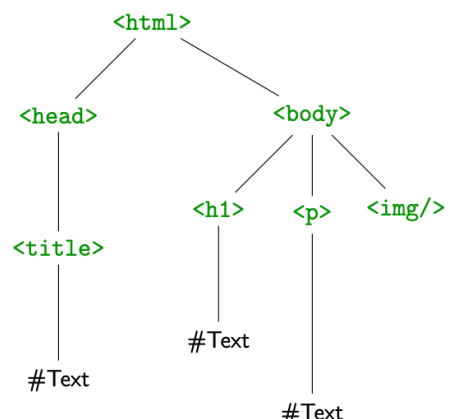
Le DOM (Document Object Mode)

On se demande comment JS connaît de trouver l'élément 'counter' avec `getElementById`. C'est avec l'aide du DOM (Document Object Mode) des pages HTML en arbre. Le DOM est une représentation objet normalisée des documents HTML (et XML), sous forme arborescente (il faut se rappeler du cours 1 sur HTML). Pour décrire les éléments de cet arbre on utilise le vocabulaire : noeud, enfant, parent.

Mais en plus, le DOM est une API qui permet d'accéder au document et de manipuler son contenu, sa structure et ses styles ; et d'interfacer un document HTML avec un langage, comme JavaScript, mais aussi Python, PHP, Java, etc. Vous pouvez trouver plus d'info dans la section *Liens outils*.

L'objet document a trois méthodes importantes :

- `getElementById( 'monID' )` qu'on a déjà vu, pour récupérer un élément avec id donné
- `getElementsByTagName( 'typeBalise' )` (attention au 'S' !) qui renvoie un tableau avec toutes les balises du type donné. Cette méthode peut s'appliquer à tout élément HTML, pas qu'à document. En mettant * en paramètre, on récupère tous les descendants.
- `getElementsByTagName( )`, de moins en moins utilisé.



Deux méthodes de document plus modernes sont : `querySelector()` et `querySelectorAll()`. Ils prennent en paramètre un sélecteur comme en CSS. Le premier renvoie la première balise correspondant, le second renvoie le tableau des correspondances.

Ex.: `let query = document.querySelector('#menu .entree span');`

Ces méthodes récupèrent un élément (noeud) ou un tableau des noeuds. Mais quoi faire avec ? En fait, avec les noeuds nous pouvons récupérer des attributs et du contenu d'une balise (élément, noeud).

Nous pouvons récupérer les propriétés d'une balise (Attributs) en utilisant `.getAttribute('nom')` et avec `.setAttribute('nom')`. Il y a même quelques attributs directs : `.attribut`, `.href`, etc.

Et l'exemple qu'on a vu est une façon de récupérer du contenu (Content). Donc `.innerHTML`, renvoie aussi les balises internes. On peut éditer ou ajouter comme une variable : `=`, `+=`, `...` (mais on peut pas ajouter `<script>` ainsi).

Le DOM a aussi des méthodes qui utilisent la structure d'arbre pour naviguer dedans, par exemple avec `.parentNode`, `.firstElementChild`, etc. Et nous pouvons même créer des noeuds (éléments), par exemple `document.createElement('a')` crée une balise type `a`, mais il faut définir ces propriétés avec `setAttribute` et l'insérer après avec `.appendChild()`.

Bon, tout ça c'est un peu trop. L'idée est que avec JavaScript nous pouvons utiliser l'API de DOM pour manipuler notre document HTML. Nous allons voir des méthodes au fur et à mesure - et il y a aussi la liste des liens utiles et l'internet !

Changer le style (CSS) avec JavaScript

JavaScript est un langage dynamique et donc c'est raisonnable de l'utiliser pour changer les feuilles de style en cascade (CSS). Il y a plusieurs façons de le faire :

Trouver l'élément dans le DOM de l'HTML et définir le ou les styles pour celui-ci un par un. Par exemple :

```
const element = document.querySelector('div');
element.style.backgroundColor = 'red';
```

Ah! Là on a utilisé `querySelector` pour trouver tous les balises avec class (CSS) `div` et nous avons utilisé le mot clé `style` pour échanger son couleur de fond. Mais attends, en CSS cette propriété avait le nom `background-color` ... Oui lorsque vous manipulez les styles CSS avec JavaScript, vous devez écrire les propriétés CSS en *camel case* (au lieu d'utiliser un tiret "-" pour séparer les mots, vous devrez écrire en majuscules la première lettre de chaque mot (ex., `backgroundColor`, `fontSize`)).

Ou si vous voulez changer plusieurs styles pour un élément, vous pouvez créer un "template" (patron) en string et le passer dans la propriété `element.style.cssText`.

```
/* Defining all our CSS styles */
const myStyles = `
```

```

width: 80%;
background-color: red;
`;
const element = document.querySelector('div');
element.style.cssText = myStyles ;

```

Après c'est aussi possible de changer la classe des éléments, appliquer un autre fichier css, etc. Vous pouvez voir plus dans la liste de liens utiles.

Remarques : Quand vous appliquez des styles spécifiques en utilisant JavaScript, ces styles ont la priorité sur les styles appliqués dans le feuille / fichier CSS et dans l'élément HTML lui-même.

Le langage JavaScript

Le langage JavaScript est assez proche du langage C/C++ (comme PHP, plus tard), c'est la même famille. JavaScript (et PHP) n'a pas de type statique. Une même variable peut être tour à tour un nombre, un string ou un booléen. Par contre contrairement à PHP les noms de variables ne doivent pas forcément commencer sur le signe \$. Du coup JavaScript aura du mal à détecter les nouvelles variables qui apparaissent dans votre code et il faudra déclarer vos variables et vos fonctions. Comme en PHP les fonctions se déclarent avec le mot clé "function".

Les variables peuvent être déclarées par trois mot-clé : `var`, `let` et `const`. A chaque fois la déclaration peut être suivie d'une affectation comme en C/C++. Vous trouverez beaucoup beaucoup de codes JavaScript qui n'utilisent que le mot-clé `var`. C'est l'ancienne solution qui a posé tellement de problèmes qu'il a fallu créer `const` et `let`. Voyons les 3 et pourquoi 99% du temps `var` devrait être remplacé par `const` ou `let`.

- `var i = 0;` déclare une variable `i` pour toute la fonction, et si dehors une fonction c'est globale). Donc si ensuite un autre bout de code utilise `i` il aura comme valeur de départ celle que vous avez laissée. Si vous pensez pouvoir utiliser `i` plus tard en récupérant la valeur que vous y avez mise vous vous trompez. L'autre bout de code (une librairie, une fonction callback sur un bouton) a très bien pu changer la valeur. Et devinez quoi le nom "i" est très commun en programmation !!!
- `let i=0;` est beaucoup mieux. Il déclare une variable locale qui ne restera que dans le bout de code actuel (une fonction, une paire d'accolades). Ça veut aussi surtout dire que si un autre `i` avait été déclaré avant/ailleurs la nouvelle variable `i` ne la remplace pas. Elle la cache seulement le temps de votre bout de code et l'autre variable `i` reviendra quand javascript sortira de votre bout de code (une fonction, une paire d'accolades). Dans le code suivant :


```

let i = "hello";
for (let i=0; i<10000; i++){
  console.log("OK "+i)
}
console.log(i);

```

 la boucle `for` affiche 10000 fois des "OK 1", "OK 2", "OK 3" puis à la fin affiche "Hello" car une fois sorti de la boucle, la variable `i` qui valait des nombres disparaît et elle arrête de cacher la variable `i` qui valait une chaîne de caractères.

- `const i=0;` marche pareil que `let i=0;` mais on ne pourra plus changer la valeur de `i`
C'est donc une sécurité, un garde-fou. Si le code plante sur "Uncaught TypeError: Assignment to constant variable." à vous de choisir entre :
 Ne pas changer la valeur
 Changer la déclaration `const` en `let`

Avant de voir les événements il nous reste à voir une autre subtilité de JavaScript. JavaScript est un langage à la fois impératif, objet et fonctionnelle !!! Alors que vous remonte tous les traumatismes du programmation impérative et programmation objet) qu'est ce que ça veut dire ?

Ca veut dire que n'importe quelle variable (déclarée avec `var`, `const` ou `let`, paramètre de fonction ou retournée par une fonction) peut être un objet ou une fonction ... Dans l'exemple suivant nous avons un script *inline* c.à.d. ajouté directement dans l'HTML dans la balise `<script>`. Ici les fonctions `el.addEventListener` a trois paramètres, une est une fonction et `setTimeout` est aussi une paramètre qui est une fonction (mais quels sont leur noms ? ...)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Titre de la page</title>

  <script>

    function init(){
      let el = document.getElementById("mabellediv");
      el.addEventListener("click",
                          function(){console.log("merci d'avoir cliqué")},
                          false);
    }

    function verifiePrenom(newPrenom){
      console.log("quelqu'un pretend s'appeler " + newPrenom);
    }

    setTimeout( function(){console.log("coucou")}, 4000);

  </script>
</head>

<body onload="init()">
  Bonjour le monde
  <div>
    Entrez votre prénom :<input type="text" name="prenom"
                              onchange="verifiePrenom(this.value)">
    <br>
  </div>
  <div id="mabellediv">hello</div>
</body>
</html>
```

Les Évènements

Regardez bien l'exemple ci-dessus et tachez de voir quels événements peuvent bien déclencher quel JavaScript. Il existe 2 mécanismes principaux.

1) Au niveau des balises on peut rajouter `onTruc="bidule( )"`. Ca veut dire que lorsque la navigateur reconnaît que l'évènement `truc` il exécute le code qui est entre guillemet. On aurait pu mettre n'importe quel code avec pleins d'instruction séparées par des `;` mais le HTML est déjà pas mal encombré. Le mieux est donc d'appeler juste une fonction pour ne pas trop encombrer le HTML.

Ce qu'il y a à savoir c'est que entre ces guillemets on a accès a la variable `this` qui représente la balise. Si le code JavaScript doit modifier la balise qui a initié l'évènement on en aura besoin. On peut passer `this` en paramètre de la fonction que l'on appelle et récupérer la valeur de la balise dans l'argument de la fonction qu'on aura déclarée. On peut aussi n'avoir besoin que de la valeur (`this.value`) de cette balise ou que de son id (`this.id`). On pourra choisir de ne passer en argument que le strict nécessaire. Tant que nous y sommes, il y a un autre objet qui existe entre ces guillemets et qui nous sera parfois utile, il s'appelle `event` et représente l'évènement. Quelles informations et fonctions importantes sont dans la balise et dans l'évènement ? C'est une bonne question mais le mieux c'est de les passer en paramètre et de les afficher dans la console pour voir ce qu'ils ont dans le ventre.

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Titre de la page</title>
  <script>
    function checkme(tag, event){
      console.log(tag)
      console.log(event)
    }
  </script>
</head>
<body>
  Entrez votre prénom :<input type="text" name="prenom"
                        onchange="checkme(this, event)"><br> </div >
</body>
</html>
```

Ah la boulet de prof a oublié de mettre des points virgules à la fin des lignes de code javascript !!! Même pas ... (question historique, nous nous essayons de toujours ajouter les ;). Et tiens, ce `input` semble être une entrée utilisateur !

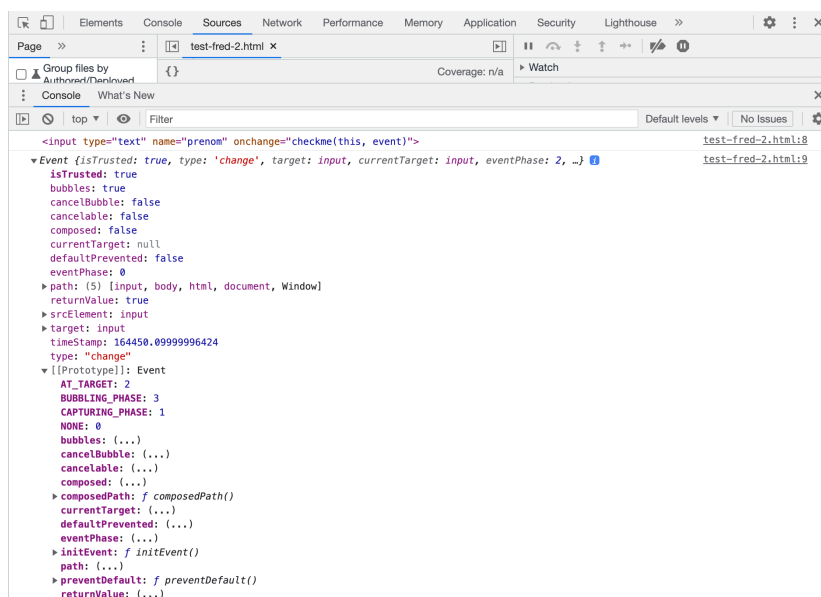
Oui effectivement, HTML a aussi des fields pour entrer du texte (`input type="text"`) comme dans l'exemple, des boutons (`input type="button"`), des outils pour sélecteur des couleurs (`input type="color"`), etc.

Chacun des ces éléments d'interface (UI elements) vient avec des événements "onTruc" (ex `onClick`, `onHover`, `onkeydown`, ...). Plus correctement il s'agit de "listeners", des fonctions qui "écoutent" pour des événements. À ces listeners nous pouvons s'abonner avec des fonctions qu'on appelle "event handlers". C'est nous pouvons créer nos propres fonctions (ici par exemple "checkme") qui sont appelées dès qu'un événement est détecté. Dans l'exemple quand l'utilisateur écrit dans la zone de texte, c'est l'événement `onchange` qui est détecté. Nous avons abonné la fonction "checkme" à cet événement, donc quand `onchange` est détecté il déclenche notre fonction `checkme`.

Il y a beaucoup des "input" (UI éléments) et des événements, impossible de les apprendre par cœur, donc merci internet !

Bon, en tout cas ce code affiche bien des choses dans la console :

Entrez votre prénom :
Anastasia



Outre le tag (la balise) et ses attributs on affiche `event`. On voit dans son `__proto__` toutes les belles fonctions qu'on pourrait appeler dessus¹.

2) Un événement important à connaître est le `onload` sur le `<body>` de la page HTML (voir le code dans la section précédente "Le langage JavaScript").

En effet cet événement se lance que quand toute la page est finie de se charger DONC quand toutes les balises sont en place. C'est important pour la seconde façon de s'occuper des événements. En effet comme vous avez pu le voir plus haut JavaScript peut aussi s'abonner *dynamiquement* du code à un événement avec `addEventListener(...)` mais pour cela il faut avoir un pointeur vers le noeud/tag/balise sur lequel on veut s'abonner.

La fonction `addEventListener()` avec trois paramètres :

- nom de l'événement, par exemple "click" (sans le *on*) ;
- fonction à exécuter (anonyme, IIFE, ou classique - quoi? il y en a plusieurs ...) ;
- (Optionnel : booléen pour phase de capture ou de bouillonnement, disons `false` par défaut).

¹ Deux fonctions intéressantes sont `event.stopPropagation()` et `event.preventDefault()` qui peuvent court-circuiter tout un tas de comportements par défaut du web.

On récupère dans l'évènement plusieurs propriétés :

`type` : `click`, `mouseover`, etc.

`currentTarget` : ce qui est l'origine de l'évènement

`target` : ce qui déclenche l'évènement (peut être un fils de `currentTarget`)

`clientX` et `clientY` : position de la souris sur l'écran, adapté à `mousemove`

`keyCode` : pour le code ASCII de la touche appuyée. Ensuite `fromCharCode()` si on veut la conversion.

`preventDefault()` pour éviter comportement par défaut

Enfin il y a `setTimeout(function(){console.log("coucou")}, 4000);` qui permet à javascript d'appeler une fonction au bout de n(=4000) millisecondes. Comme javascript n'est pas multi-threadé (enfin sans bidouille) cela permet de déclencher du code (comme les Timer en java). Il y a aussi `setInterval()` dans la même veine qui va déclencher des événements à intervalle régulier (plus que un).

Javascript permet aussi de faire tout cela et pour les cas simples c'est même plus rapide. Les adeptes du JavaScript ont toutefois beaucoup de mal à convaincre les adeptes de tel ou tel framework que le JS pur c'est bien aussi, du coup il ont inventé un fake framework : `VanillaJS` (<http://vanilla-js.com/>) qui ne fait rien. En montrant des exemples et des benchmarks de performance ils arrivent parfois à convaincre des brebis égarées de revenir vers le JavaScript pur. Les développeurs web ne jurent au final que par un seul mantra : "Je cherche du code qui fait ce que je veux avec Google, je copie-colle les codes que je trouve dans mon code en adaptant un peu, si ça marche je ne touche plus à rien".

Vous trouverez beaucoup de codes sur internet et beaucoup d'exemple utiliseront telle ou telle librairie. Si à chaque fois que vous récupérez un bout de code vous devez importer une librairie vous aurez au final un code monstrueux qui importe beaucoup de choses, qui est long à charger, lent à s'exécuter et que les utilisateur fuiront. Mais le pire c'est encore d'utiliser une super librairie que vous utilisez à pleins d'endroits stratégiques dans votre application web ... puis un jour les développeurs de cette super librairie arrêtent de la mettre à jour, les navigateurs continuent d'évoluer et les librairies ne fonctionnent plus ou mal ou mal avec une autre librairie dont vous avez besoin. Moi je vous propose de voir au future JQUERY car elle est TRES utilisée (peu de risque qu'elle devienne obsolète à moyen terme) et qu'elle réutilise le savoir-faire des sélecteurs CSS qui sont très pratiques.

TypeScript : TypeScript est un langage de programmation qui ajoute des fonctionnalités supplémentaires à JavaScript. JavaScript n'a jamais été conçu pour piloter des applications front-end et back-end complexes. Le problème est que JS permettait de faire très facilement des erreurs de programmation, au particulière liées aux typage. TypeScript a été développé spécifiquement pour résoudre ces problèmes tout en restant compatible avec les environnements JavaScript existants. C'est une langage typé :

<https://www.typescriptlang.org/>

À partir d'ici une référence de JavaScript.

JavaScript et Déclaration des variables

- `let myVariable;` une variable n'est déclarable qu'une seule fois au sein du même bloc. Dans notre exemple, déclarer plusieurs variables a donnera lieu à une erreur. Cela commence à devenir le standard.
- `const a="Info";` une variable déclarée `const` ne peut être modifiée.

Dans les deux cas, `let` et `const` donnent une portée limitée au bloc dans lequel sont déclarées les variables.

- À éviter, mais on trouve aussi souvent :
`var myVariable1, myVariable2 = 4, myVariable3;`
Attention à la portée : si variables déclarées sans `var`, elles sont globales peu importe où elles sont déclarées ! Et risquent être modifiées (ex par les librairies). Avec `var` ils sont déclarés dans toute la fonction (et donc si dehors une fonction pour la vie du javascript ...)

Types

Différents types :

- Numérique : 5; 5.5; 3.65e+5; 0x391;
Opérateurs : +, -, *, /, %, +=, -=, *=, /=, %+=, ++, --
- String : entre apostrophes ou guillemets, attention à l'échappement 'c'est un exemple'
Opérateur + pour concaténation
Méthodes : `.length`, `.toUpperCase()`, `.toLowerCase()`
- Booléens : `true` ou `false`
`typeof` (ex : `typeof number` ; `undefined` sinon)

Comparaison entre types :

- + entre numérique et string donne string
`'J'ai ' + 20 + 'ans'; text = number1 + " " + number2;`
- Dans l'autre sens, `number = parseInt(text)`
- = pour affectation, == pour comparaison non typée (ex : `'10' == 10` est vrai), === pour comparaison typée

Structures de contrôle

Opérateurs de comparaison :

`==, !=, ===, !==, >, >=, <, <=, &&, ||, !`

Structures conditionnelles :

- `if(){...} else if (...) {...} else{...}`
- Fonction `confirm()` renvoie un booléen
- Ternaires : `let endMessage = adult ? '18+' : '-18';`
- `switch () case: break; default:`
- `if (variable) : false` ssi variable nulle ou string vide ou non définie

Boucles :

- `while(condition) {instructions;}`
- `do{instructions;} while(condition)`
- `for (init.; cond.; incr.) {instructions;}`
Ex: `for (let i = 0; i < 5; i++){alert('Itération n' + i);}`
- `break;` et `continue;`

Fonctions

Syntaxe classique :

```
function maFonction(arguments) { instructions;}
```

```
ex: function essai(arg1, arg2) {}
```

Portée des arguments et des variables : locale

Possibilité variables facultatives à la fin qu'on ne complète pas, on donne comportement par défaut avec `if` et `typeof` `== 'undefined'`
`return`, fin de fonction

Fonction anonyme :

```
let ditBonjour = function(){alert('Bonjour!')};
```

```
ditBonjour();
```

Ici on "stocke" le code d'une fonction dans une variable - cette fonction n'a pas de nom.

IIFE (Immediately-Invoked Function Expression) :

Ceci est une variation de la fonction anonyme ou en une seule étape on : écrit le code (dans la première parenthèse) et on l'appelle (indiqué par la deuxième parenthèse).

```
(function() {instructions;} )();
```

Attention à la portée : si variables déclarées sans `var` / `let`, elles sont globales peu importe où elles sont déclarées ! Si variables déclarés avec `var` peuvent changer ensuite car `var` déclare pour toute la fonction. À privilégier `let` qui déclare que dans le bloc.

Tableaux / "Arrays"

- Syntaxe `var monTableau = ['nom0', 'nom1', 3, 'nom2'];`
Rq : pas tous de même type nécessairement
- Indexé à 0, `monTableau[i]`, `monTableau.length`, `alert(monTableau)`
- `push(1,2)` ajoute à la fin, `unshift()` au début
- `pop()` retire à la fin, `shift()` au début
- String et tableaux : `phrase.split(' ')` coupe phrase selon le séparateur espace et crée un tableau. Contraire : `join()`
- Dictionnaires :
`let famille = {soi : 'Daniel', soeur : 'Sarah'}`
`famille.sister` équivalent à `famille['sister']`
Ajout : `famille.oncle = 'Jerome'`
- Parcours avec `for in` :
`for (let id in family){alert(family[id]);}`

Accès au DOM

1) Accéder : L'objet `document` a trois méthodes :

- `getElementById('monID')` pour récupérer un élément avec id donné
- `getElementsByTagName('typeBalise')` (attention au 'S' !) : renvoie un tableau avec toutes les balises du type donné.
Peut s'appliquer à tout élément HTML, pas qu'à document
En mettant * en paramètre, on récupère tous les descendants
- `getElementsByTagName()`, de moins en moins utilisé

Deux méthodes de `document` plus modernes : `querySelector()` et `querySelectorAll()`. Prend en paramètre un sélecteur comme en CSS. Le premier renvoie la première balise correspondant, le second renvoie le tableau des correspondances.

Ex. : `let query = document.querySelector('#menu .entree span');`

2) Récupérer : Pour les Noeuds du DOM on peut récupérer :

- Attributs :
 - propriétés d'une balise `.getAttribute('nom')`, `.setAttribute('nom')`

- Quelques attributs directs : `.attribut`, `.href`, etc.
- Marche pas toujours : `classe` : `.className` ; `for` (des formulaires) : `.htmlFor`
- Si plusieurs classes, méthode `.classList` :
`div.classList.add('new-class');`,
de même `.remove()`, `.contains()` (booléen)
- Contenu :
 - `.innerHTML`, renvoie aussi les balises internes
On peut éditer ou ajouter comme une variable : `=`, `+=`, ... (mais on peut pas ajouter `<script>` ainsi)
 - `.textContent` ne renvoie pas les balises internes. Attention pas sur tous navigateurs ! Cf. <https://openclassrooms.com/fr/courses/1916641-dynamisez-vos-sites-web-avec-javascript/1918282-manipuler-le-code-html-partie-1-2>

3) Propriétés du Nœud :

- `.nodeType` : entier (1 : Nœud élément, 2 : Nœud attribut, 3 : Nœud texte, etc.)
- `.nodeName` : nom de l'élément, en majuscule (attention !)
- `.nodeValue` : contenu d'un nœud textuel (si ce n'est pas le cas, descendre aux enfants)

4) Se déplacer :

- `.parentNode`
- `.firstChild`, `.lastChild`, `.childNodes` (tableau), `.nextSibling`, `.previousSibling`
- Attention si on veut que vraies balises, ou attention au caractère saut de ligne : `.firstElementChild`, `.lastElementChild`

5) Création :

- `document.createElement('a')` crée balise type a, pas encore inséré Nœud textuel `createTextNode()`
- Le définir avec `.setAttribute`, etc.
- Insérer `.appendChild()` :
`document.getElementById('monId').appendChild(newLink);`

6) Autres méthodes :

- `.cloneNode()` avec paramètre `true` ou `false` si on veut cloner enfants et attributs (ne marche pas pour événements)
- `.replaceChild()`
- `.removeChild()`
- `.insertBefore()`

Liste des événements

click : cliquer (appuyer puis relâcher) sur l'élément

dblclick : double-cliquer sur l'élément

mouseenter : faire entrer le curseur sur l'élément

mouseover : faire entrer le curseur sur l'élément ou un de ses fils

mouseout : faire sortir le curseur de l'élément

mouseup : relâcher le bouton gauche de la souris sur l'élément

mousemove : faire déplacer le curseur sur l'élément

keydown : appuyer (sans relâcher) sur une touche de clavier sur l'élément

keyup : relâcher une touche de clavier sur l'élément

keypress : frapper (appuyer puis relâcher) une touche de clavier sur l'élément

focus : cibler l'élément

blur : annuler le ciblage de l'élément

change : changer la valeur d'un élément spécifique aux formulaires

select : sélectionner le contenu d'un champ de texte

Trois façons d'invoquer les événements

1) Sans DOM, directement dans l'HTML : avec on + nom événement :

Ex : `<span onclick="alert('Hello !');">Cliquez-moi !</span>`

2) Avec DOM-0, propriétés des attributs, toujours avec on + nom évt :

Ex :

```
<span id="clickme">Cliquez -moi !</span> <script>
var element = document.getElementById('clickme '); element.onclick = function() {
alert("Vous m'avez cliqué !"); };
</script>
```

3) DOM-2, objet Event, fonction `addEventListener()` avec trois paramètres :

nom de l'évènement (sans le on)

fonction à exécuter (anonyme, IIFE, ou classique)

Optionnel : booléen pour phase de capture ou de bouillonnement

On peut aussi retirer avec `removeEventListener()` + même arguments (problème des IIFE ici...)

On récupère les informations sur l'évènement.

```
DOM-0 : element.onclick = function(e){alert(e.type)};
```

```
DOM-2 : element.addEventListener('click', function(e)
{alert(e.type)});
```

On récupère dans e l'évènement. Propriétés :

`type` : click, mouseover, etc.

`currentTarget` : ce qui est l'origine de l'évènement

`target` : ce qui déclenche l'évènement (peut être un fils de `currentTarget`)

`clientX` et `clientY` position de la souris sur l'écran, adapté à `mousemove`

`keyCode` pour le code ASCII de la touche appuyée. Ensuite `fromCharCode()` si on veut la conversion.

`preventDefault()` pour éviter comportement par défaut

Liens utiles

1. l'API du DOM : <https://dom.spec.whatwg.org/>
2. Les objets du DOM : https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
3. Changer CSS avec JavaScript (en Anglais) <https://alvarotrigo.com/blog/change-css-javascript/>
4. Cours en Français par P.Giraud <https://www.pierre-giraud.com/javascript-apprendre-coder-cours/>
5. Internet ...
6. Si vous voulez aborder TypeScript : <https://www.typescriptlang.org/>